COMPUTER SHOP MANAGEMENT SYSTEM

RONALD ALLAN ZAMORA

JAMES RYAN V. GESTA

ANSARY M. ASIS

BHONG S. KILATON

AL KENNETH M. UNTALAN

KARL JOROSH C. PACULAN

1. Overview

This document describes how the project was managed from vision to defense-ready submission. It covers the development methodology and why it fits the evidence, the phases actually followed with their real deliverables, team roles, how scope changes were controlled, the risks encountered with their mitigations, and the quality gates enforced before any work was accepted. It is a management record, not a technical manual: for code behavior see CODE_WALKTHROUGH.md, for the database see docs/Database_Doc.md, for structures see docs/DataStructures_Doc.md, for OOP see docs/OOP_Doc.md, and for deployment plus defense Q&A see TECHNICAL_DOCUMENTATION.md.

Management scope in one paragraph: a single developer carried every role, so process weight was kept deliberately light — no sprint ceremonies, no ticket tracker, no status meetings. Control came from four habits instead: work was cut into reviewable stages, nothing advanced without explicit approval, every change was verified the same way it would be demonstrated, and the working database was never used as a test bed. Those habits are the entire process described below, stated plainly so a panel can check each claim against the repository history.

2. Methodology: Iterative and Incremental

2.1 Definition

Iterative and Incremental development builds the system in small working wholes rather than in one big delivery. Each iteration adds or refines a slice of functionality, and each increment leaves behind a compilable, runnable system. Planning, building, and verifying repeat every cycle instead of happening once each.

2.2 Why This Methodology Fits the Evidence

- **Increments, not one big bang:** login plus database wiring came first, then the counter dashboard, then the customer timer, then auto-stop, broadcast, the DAO refactor, data structures, and finally documentation. Every stage ended with a compiling program — at no point did the repository hold a half-wired system that could not run.
- **Iteration within increments:** the login page went from split-screen to centered card to compact to rose-fill across passes; the auto-stop defect went through investigate-first, then fix; documentation went through notation precision passes. Requirements were refined by review, never frozen upfront — each pass kept what worked and changed only what the review flagged.
- **Explicit review gates:** nearly every stage ended with a show-then-stop rule — demonstrate the result and wait for confirmation before the next stage began. That gate is the project's real quality mechanism, and it is visible in the work log as a repeated approve-then-proceed rhythm rather than a single final inspection.

2.3 Why Not the Alternatives

- **Waterfall is falsified by the record:** scope changed mid-stream (role-based login added, products/transactions dropped for customers/session_log, broadcast added late, password hashing introduced during refactoring). A frozen-requirements model cannot honestly describe this history — any phase diagram drawn after the fact showing one-way arrows would be fiction, because design decisions demonstrably flowed backward from testing more than once.
- **Agile/Scrum would be dishonest:** no sprints, backlogs, standups, or ceremonies existed. Claiming them would invent process for the panel to question. What the project did borrow

from Agile thinking is the working-software measure of progress: a stage counted as done only when it compiled and behaved, never when a document said so.

- **Prototyping is true but partial:** the NetBeans Design-first habit (GUI laid out visually, then wired) is real, but it describes only interface construction, not the delivery model. It is recorded here as a supporting technique, not the methodology. Throwaway prototyping never happened either — every prototype evolved directly into the shipped code.

2.4 Methodology Comparison

Criterion	Waterfall	Agile/Scrum	Iterative/Incremental (chosen)
Requirements timing	Frozen upfront	Evolving backlog	Evolved per review gate
Delivery shape	Single final release	Sprint demos	Working system after every stage
Fit for solo academic work	Poor: no room for course corrections	Poor: ceremonies with no team	Good: light gates, real demos
Evidence in this repo	None — scope changed repeatedly	None — no sprints exist	Stage history, review rhythm, refinement passes

3. Phases and Actual Deliverables

Phase	Our Stages	Deliverables Produced
Inception	Project vision, rubric study	Scope statement, pisonet-replacement goal, 100-point rubric mapping
Elaboration	Database design, login	java_database schema (3 tables), Login.java, connectDB.java
Construction 1	Counter + customer	Dashboard.java (Start/Extend/Stop), CustomerTimer.java (live countdown + popups)
Construction 2	Automation + messaging	Auto-stop expiry, broadcast flags, sales total
Construction 3	Refactor + structures	DAO layer, SHA-256 login, Session POJO, Stack/sorts/binary search, Undo/Sort/Find/Logs buttons
Construction 4	Schema evolution	customers, session_log, customer-linked sessions, View Logs, card names

Phase exits were decided by demonstration, not by calendar: a phase ended when its deliverables compiled, behaved as specified, and were shown working — which is why some phases contain several internal iterations (the login page alone passed through four visual revisions inside Construction 1) while others, like the DAO refactor, passed in a single straight pass once the mapping tables were approved.

Each phase ran the same entry/exit contract: entry required an approved plan with verified anchors (live file reads, never memory); exit required a clean compile, a live demonstration of behavior, and explicit approval before the next phase began. A phase that failed any clause simply repeated — the auto-stop defect, for example, cycled investigate → fix → verify twice before it was accepted.

Transition: Packaging + defense — setup.sql, CODE_WALKTHROUGH.md, docs/ series, TECHNICAL_DOCUMENTATION.md

4. Team Roles and Responsibilities

Role	Responsibilities	Holder
Project Manager	Stage sequencing, scope rulings, review gates, submission checklist	[Name]
Developer	GUI, JDBC, DAO, and data-structure implementation	[Name]
Database Administrator	Schema design, normalization, seeds, setup.sql, phpMyAdmin verification	[Name]
QA Tester	Stage-5 and final testing passes, defect reports (e.g. stuck IN USE, empty-shop broadcast)	[Name]

One contributor may hold several roles; the table records duties, not headcount. No teammates are invented here. In practice the roles collapsed in a fixed order per task: DBA work (schema reads and scratch-database tests) came before Developer work (edits), and QA work (compile

plus behavior verification) closed every task — so even solo, the duties stayed separated in time if not in person.

Duties in practice:

- Project Manager: froze stage scope in writing before any edit, ran the approve-then-proceed gate, owned the submission checklist.
- Developer: wrote GUI, JDBC, DAO, and data-structure code; never edited a file without a live read in the same task.
- Database Administrator: designed and normalized the schema, wrote seeds, verified every migration on a scratch database first, and confirmed the working database was never used as a test bed.
- QA Tester: ran the Stage-5 and final testing passes, filed the defect reports that drove real fixes (stuck IN USE cards, empty-shop broadcast behavior).

5. Change Control

Scope changes were accepted only with an explicit ruling, a code-and-schema impact check, and a same-task verification run. Real examples from this project:

- products/transactions removed for customers/session_log: impact checked (only seed and schema work affected, no GUI yet depended on the dropped tables), then rebuilt as customer-linked sessions with logging.
- Broadcast added late: required two new pcs columns, so it shipped as an ALTER plus a commented upgrade block in setup.sql rather than a silent schema fork — old and new databases stay distinguishable on purpose.
- Password hashing introduced during refactoring: forced a credential migration (stored digests updated) plus a connectivity guard, instead of leaving mixed plaintext/hash rows.
- Rejected or deferred without implementation: receipt printing, membership accounts, multi-branch support, email/priority broadcast features — recorded in the vision's out-of-scope list, never half-built. A deferred item stays deferred until a new ruling reopens it; none was reopened.

Verification outcomes for the accepted changes: the rebuilt sessions flow compiled clean with all foreign keys intact; the broadcast ALTER applied without touching existing rows; the hashing

migration was proven by logging in with the old plaintext password (which the app hashes at comparison time). Each outcome was demonstrated, not assumed.

6. Risk Register

Risk	Evidence	Mitigation Actually Used
Stale builds hiding fixes	Auto-stop "fix" present in source but reported not working	Clean rebuilds plus verifying behavior, not just code presence — the fix was confirmed by watching a forced-expired session flip to AVAILABLE, not by re-reading the method
Clock disagreement	Java display clock vs MySQL NOW()	All writes and expiry comparisons use the database clock on both sides
Seed FK failures	Transaction seeds pointed at nonexistent sessions	Read-only schema inspection first; seeds adjusted to existing rows
Secret exposure	Deployment token shared in plaintext chat	One-shot auth headers, nothing persisted, token revoked after use
Dead database at demo	XAMPP off, wrong DB selected	Friendly dialogs on every user action; quiet background ticks; scratch-DB testing that never touched the working database
Scope creep into docs	Walkthrough line-number drift after edits	Every citation re-verified with grep before each documentation task closed

Accepted residual risks (known, unmanaged by choice): single-developer bus factor — all process knowledge sits with one person, mitigated only by this documentation series; no automated test suite — verification is manual per stage, sufficient at this scale but not beyond it; single-machine XAMPP hosting with no backup routine — acceptable for a classroom deployment, flagged here so it is a conscious choice rather than an oversight.

7. Quality Gates

No stage closed until all of its gates passed — the same gates, every time:

- Full-project javac exits 0 with no new warnings. The three remaining this-escape notes are inherent to the NetBeans form pattern and predate every stage, so "no new warnings" is the enforced bar, not "zero warnings".

- Read-before-edit: no file was modified without a live read in the same task — the rule that caught the stale-reconstruction mistake before it shipped.
- Show-then-stop: results displayed and explicitly approved before the next stage began. Skipped exactly never; even one-line fixes were shown back.
- Database work verified on a scratch database first; the working database was never used as a test bed. The scratch database was dropped after each verification, leaving zero trace.
- Documentation cites real names, real line numbers, and real SQL — re-checked, never reconstructed from memory. Line-number drift (from intervening edits) was caught twice by re-grepping and corrected before delivery.

8. Defense Questions This Document Answers

- "What methodology did you use, and why?" → Iterative/Incremental: staged working deliveries with review gates; Waterfall is falsified by mid-stream scope changes, Scrum ceremonies never existed.
- "How did you control scope changes?" → Explicit rulings with impact checks, e.g. the products-to-customers rebuild and the broadcast ALTER with its commented upgrade block.
- "What was your biggest risk, and how did you handle it?" → Stale-build confusion and clock disagreement, handled with clean rebuilds plus behavior verification and database-clock-only timestamps.
- "One developer held all roles — how is that managed?" → Duties separated in time per task (DBA reads first, Developer edits, QA verifies last), recorded in Section 4.
- "What would you do differently with a team of five?" → Assign the four Section 4 roles to different people, keep the same gates, and add a shared issue list — the process scales without changing shape. Line-number drift (from intervening edits) was caught twice by re-grepping and corrected before delivery.

OCTOBER 2026